

UNIVERSIDAD DE LAS AMÉRICAS PUEBLA

ESCUELA DE INGENIERÍA

DEPARTAMENTO DE COMPUTACIÓN, ELECTRÓNICA
Y MECATRÓNICA



Lab report #6

Course: Digital design LRT2022-7

Equipo: 2

183339 Juan Pablo Lopez Moreno LMT
185406 Amy Marianee Ramírez Sánchez LMT

November 26th, 25, San Andrés Cholula, Puebla

1 Abstract

This report will present the procedure utilized during our lab experimentation, for the sixth lab report. The practice ensured that we continued developing the usage of FPGAs, as well as, continuing the use of VHDL as a programming language. The practice focused on the development and testing of flip flops, ensuring the understanding of these, and the implementation of these components using a clock.

2 Introduction

Flip-flops are pivotal components in digital electronics, serving as fundamental units for storing binary data. They are essential in sequential logic circuits where information needs to be stored, processed, and outputted in a controlled manner. [1] Components like flip-flops are the staples of memory elements, being some of the most frequently known uses for these their appearance in registers, counters, and memory devices. Flip-flops are bistable devices, meaning they have two stable states, typically represented as 0 and 1. They can maintain their state indefinitely until triggered to change by an input signal, making them ideal for storing bits of information. These Flip flops can be classified into several types, however, for the practical development in the lab, we will only be focusing on the following variation:

- SR Flip-Flop: The Set-Reset flip-flop has two inputs, S (set) and R (reset), and can store one bit of data. It toggles between two stable states based on the input signals
- D Flip-Flop: The Data flip-flop captures and holds the value of its D (data) input when triggered by a clock signal. It is widely used for synchronizing data transfers.
- JK Flip-Flop: Named after its inventors, Jack Kilby and Jerry Merryman, the JK flip-flop combines the features of the SR and T flip-flops, offering more flexibility in circuit design.
- T Flip-Flop: The Toggle flip-flop changes its state (either 0 or 1) with each clock pulse, effectively toggling between its two states.

[1] The very relevance of these components in digital systems have been fundamental in the development of technologies for many years, the reason being, the capabilities for storing data, making sequential systems possible.

3 Objectives

The objectives of this lab are:

- Program and simulate Flip-flops in VHDL
- Implement Synchronous FF on Basys 3 board.

For the purpose of this lab, we must understand the following concepts:

- Sequential Logic Circuits

- VHDL Programming
- FPGA Implementation

Of which, we must begin explaining A sequential logic circuit is a type of digital circuit that can store and remember information or data between clock cycles. It uses a memory element, such as a flip-flop or register, to store and update the state of the circuit based on the input and previous state. These circuits can perform a variety of functions, such as counting, shifting, and detecting and responding to specific input patterns. [2] VHDL, which has been used in previous labs, is a hardware description language that allows designers to describe the behavior and structure of digital systems. VHDL is widely used for designing FPGAs and ASICs (Application-Specific Integrated Circuits) due to its ability to model complex digital systems at various levels of abstraction.[3] Finally,field programmable gate array (FPGA) is a versatile type of integrated circuit, which, unlike traditional logic devices such as application-specific integrated circuits (ASICs), is designed to be programmable (and often reprogrammable) to suit different purposes, notably high-performance computing (HPC) and prototyping.[4] FPGA implementation involves programming the FPGA device with the designed VHDL code to realize the desired combinational logic circuit. This process includes synthesis, mapping, placement, and routing of the design onto the FPGA fabric.

4 Methodology

The methodology applied in this laboratory practice was structured into three main phases: logical analysis, simulation in EDA Playground, and physical implementation on the Basys 3 board.

4.1 Logical Analysis Phase

During this first stage, the following activities were carried out:

1. The truth tables corresponding to the half-subtractor, multiplexer, magnitude comparator, and demultiplexer circuits, all of one bit, were obtained.
2. Based on the truth tables, the corresponding Karnaugh maps were developed in order to simplify the Boolean expressions and obtain the minimal logical function for each circuit.
3. Finally, the obtained results were documented for use in the subsequent simulation and programming phases.

4.2 EDA Playground Simulation Phase

In the EDA Playground digital simulation environment, the following procedures were performed:

1. The half-subtractor, multiplexer, magnitude comparator, and demultiplexer circuits were programmed using the VHDL language.

2. Testbenches were developed to verify the behavior of each circuit under all possible input combinations.
3. The results obtained from the simulations were analyzed and compared with the previously developed truth tables and simplified functions, ensuring the correct logical implementation of each design.

4.3 Basys 3 Board Implementation Phase

Finally, the physical implementation of the circuits was carried out on the Basys 3 board, following the steps described below:

1. The code for each circuit was synthesized, and the corresponding bitstream files were generated.
2. Pin constraints (.xdc file) were assigned to map the logical signals to the physical components of the board (switches, LEDs, and displays).
3. The designs were loaded onto the Basys 3 board, verifying their correct operation through practical input and output tests.

5 Results

In this lab, the physical circuits were not built, but based on certain truth tables, EDASWaves were generated in order to accomplish every single sequential circuit given:

5.1 RS Flip-Flop

For the RS flip-flop, the following truth table was used to generate the EDA Code:

Table 1: RS Flip-Flop Truth Table

Inputs		Outputs		
S	R	Q_{n+1}	Q'_{n+1}	Function
0	0	Q_n	Q'_n	Hold (No Change)
0	1	0	1	Reset
1	0	1	0	Set
1	1	×	×	Invalid (Forbidden)

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 1: Code 1: For exercise 1

```
library IEEE;
use ieee.std_logic_1164.all;

entity sr_sync is
  port(
```

```

        clk : in std_logic;
        s   : in std_logic;
        r   : in std_logic;
        Q   : out std_logic
    );
end sr_sync;

architecture behavioral of sr_sync is
    signal Q_reg : std_logic := '0';
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if (s = '0' and r = '0') then
                Q_reg <= Q_reg;
            elsif (s = '1' and r = '0') then
                Q_reg <= '1';
            elsif (s = '0' and r = '1') then
                Q_reg <= '0';
            elsif (s = '1' and r = '1') then
                Q_reg <= 'X';
            end if;
        end if;
    end process;

    Q <= Q_reg;
end behavioral;

```

With it's appropriate testbench:

Código 2: Code 2: Testbench for exercise 1

```

library IEEE;
use ieee.std_logic_1164.all;

entity tb_sr_sync is
end tb_sr_sync;

architecture behavior of tb_sr_sync is

    component sr_sync
        port(
            clk : in std_logic;
            s   : in std_logic;
            r   : in std_logic;
            Q   : out std_logic
        );

```

```
end component;

signal clk : std_logic := '0';
signal s   : std_logic := '0';
signal r   : std_logic := '0';
signal Q   : std_logic;

constant clk_period : time := 10 ns;

begin

    uut : sr_sync
        port map(
            clk => clk,
            s   => s,
            r   => r,
            Q   => Q
        );

    — Generador de reloj
    clk_process : process
    begin
        clk <= '0';
        wait for clk_period/2;
        clk <= '1';
        wait for clk_period/2;
    end process;

    stim_proc : process
    begin
        — Test 1: Retener
        s <= '0'; r <= '0';
        wait for clk_period;

        — Test 2: Reset
        s <= '0'; r <= '1';
        wait for clk_period;

        — Test 3: Retener
        s <= '0'; r <= '0';
        wait for clk_period;

        — Test 4: Set
        s <= '1'; r <= '0';
        wait for clk_period;
```

```

-- Test 5: Retener
s <= '0'; r <= '0';
wait for clk_period;

s <= '1'; r <= '1';
wait for clk_period;

wait;
end process;

end behavior;

```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

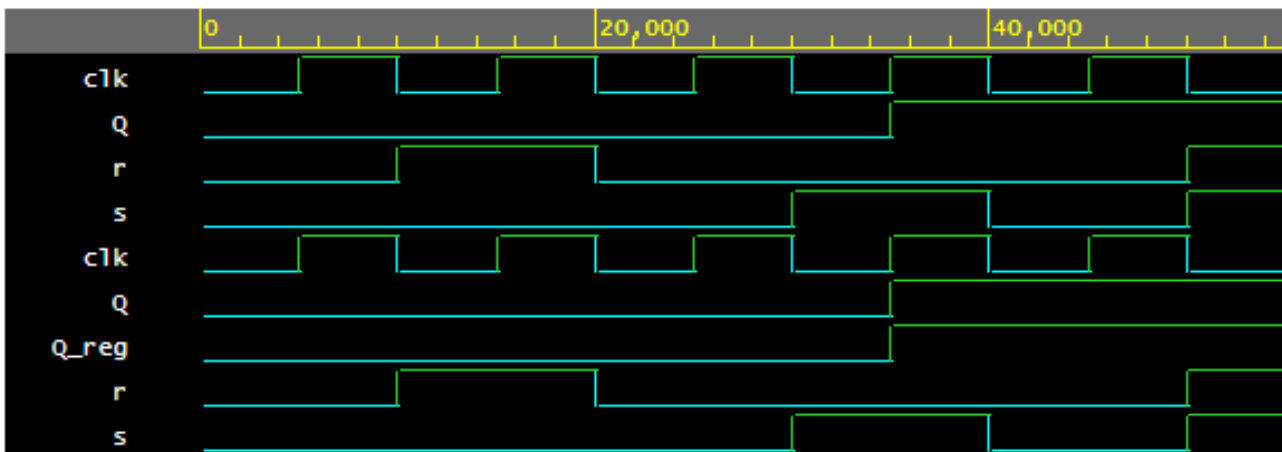


Figure 1: Wave generated for the RS flip-flop.

Finally, the code was synthesized and implemented in the Basys 3 board, obtaining the expected results.

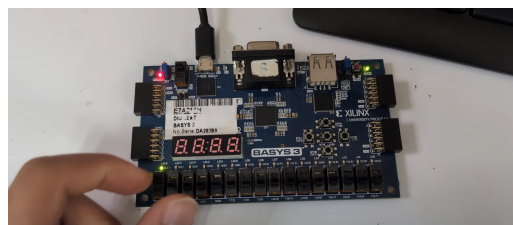


Figure 2: Evidence of the functioning of the RS flip-flop in the Basys 3 board.

5.2 JK Flip-Flop

For the JK flip-flop, the following truth table was used to generate the EDA Code:

Based on this truth table, the following Code was generated in EDA Playgrounds:

Table 2: JK Flip-Flop Truth Table

Inputs		Outputs		
J	K	Q_{n+1}	Q'_{n+1}	Function
0	0	Q_n	Q'_n	Hold (No Change)
0	1	0	1	Reset
1	0	1	0	Set
1	1	Q'_n	Q_n	Toggle

Código 3: Code 1: For exercise 2

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity jk_flipflop is
    Port (
        clk : in  STD_LOGIC;
        J   : in  STD_LOGIC;
        K   : in  STD_LOGIC;
        Q   : out STD_LOGIC
    );
end jk_flipflop;

architecture Behavioral of jk_flipflop is
    signal q_int : STD_LOGIC := '0';
begin

    process(clk)
    begin
        if rising_edge(clk) then
            case (J & K) is
                when "00" => q_int <= q_int;           -- No cambia
                when "01" => q_int <= '0';             -- Reset
                when "10" => q_int <= '1';             -- Set
                when "11" => q_int <= not q_int;        -- Toggle
                when others => null;
            end case;
        end if;
    end process;

    Q <= q_int;

end Behavioral;

```

With it's appropriate testbench:

Código 4: Code 2: Testbench for exercise 2


```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_jkff is
end tb_jkff;

architecture Behavioral of tb_jkff is

    signal clk : STD_LOGIC := '0';
    signal J    : STD_LOGIC := '0';
    signal K    : STD_LOGIC := '0';
    signal Q    : STD_LOGIC;

    — Instancia del JK
    component jk_flipflop
        Port (
            clk : in  STD_LOGIC;
            J   : in  STD_LOGIC;
            K   : in  STD_LOGIC;
            Q   : out STD_LOGIC
        );
    end component;

begin

    uut: jk_flipflop
        port map (
            clk => clk,
            J   => J,
            K   => K,
            Q   => Q
        );

    — Clock 10 ns
    clk <= not clk after 5 ns;

    stim_proc: process
    begin
        — 00: Hold
        J <= '0'; K <= '0';
        wait for 20 ns;

        — 10: Set
        J <= '1'; K <= '0';
        wait for 20 ns;
```

```

    -- 01: Reset
    J <= '0'; K <= '1';
    wait for 20 ns;

    -- 11: Toggle
    J <= '1'; K <= '1';
    wait for 40 ns;

    wait;
end process;

end Behavioral;

```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

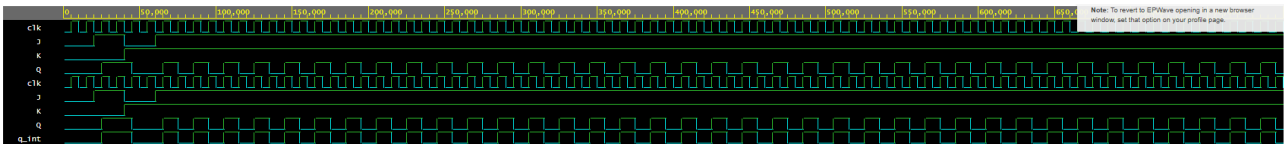


Figure 3: Wave generated for the JK flip-flop.

Finally, the code was synthesized and implemented in the Basys 3 board, obtaining the expected results.

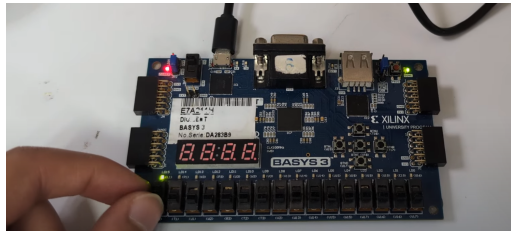


Figure 4: Evidence of the functioning of the JK flip-flop in the Basys 3 board.

5.3 D Flip-Flop

For the D flip-flop, the following truth table was used to generate the EDA Code:

Table 3: D Flip-Flop Truth Table

Input	Outputs		
	D	Q_{n+1}	Q'_{n+1}
0	0	1	Reset (Clear)
1	1	0	Set

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 5: Code 1: For exercise 3

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity d_flipflop is
    port(
        clk : in std_logic;
        d   : in std_logic;
        q   : out std_logic
    );
end d_flipflop;

architecture behavioral of d_flipflop is
    signal q_int : std_logic := '0';
begin
    process(clk)
    begin
        if clk = '1' then                -- cuando clk=1
            q_int <= d;                  --  $Q(n+1) = D$ 
        end if;                        -- cuando clk=0, no cambia
    end process;

    q <= q_int;
end behavioral;

```

With it's appropriate testbench:

Código 6: Code 2: Testbench for exercise 3

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_dff is
end tb_dff;

architecture behavior of tb_dff is

    component d_flipflop
        port(
            clk : in std_logic;
            d   : in std_logic;
            q   : out std_logic
        );
    end component;

    signal clk : std_logic := '0';
    signal d   : std_logic := '0';

```

```

    signal q    : std_logic;

begin

    uut: d_flipflop
        port map(
            clk => clk ,
            d   => d ,
            q   => q
        );

    -- reloj
    clk_process : process
    begin
        clk <= '0';
        wait for 5 ns;
        clk <= '1';
        wait for 5 ns;
    end process;

    stim_proc : process
    begin
        d <= '1';
        wait for 10 ns;    -- Q se mantiene

        d <= '0';
        wait for 10 ns;

        d <= '1';
        wait for 10 ns;

        wait;
    end process;

end behavior;

```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

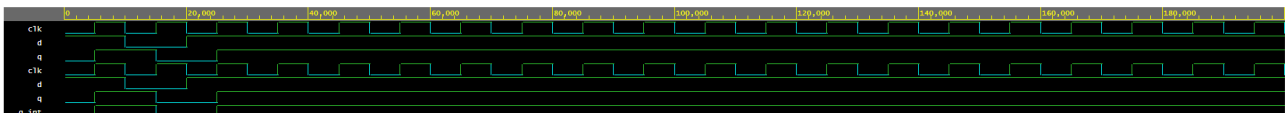


Figure 5: Wave generated for the D flip-flop.

Finally, the code was synthesized and implemented in the Basys 3 board, obtaining the expected results.

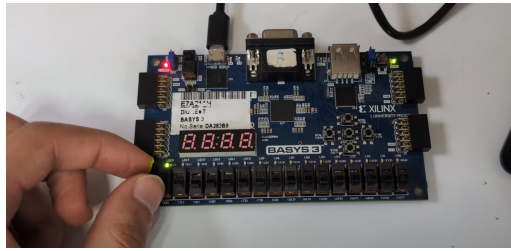


Figure 6: Evidence of the functioning of the D flip-flop in the Basys 3 board.

5.4 T Flip-Flop

For the T flip-flop, the following truth table was used to generate the EDA Code:

Table 4: T Flip-Flop Truth Table

Input T	Outputs		
	Q_{n+1}	Q'_{n+1}	Function
0	Q_n	Q'_n	Hold (No Change)
1	Q'_n	Q_n	Toggle

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 7: Code 1: For exercise 1

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity t_flipflop is
    port(
        clk : in std_logic;
        t   : in std_logic;
        q   : out std_logic
    );
end t_flipflop;

architecture behavioral of t_flipflop is
    signal q_int : std_logic := '0';
begin

    process(clk)
    begin
        if clk = '1' then
            if t = '1' then
                q_int <= not q_int;
            end if;
            -- si T=0 no cambia
        end if;
    end process;
end architecture;

```

```

        — si clk=0 no cambia
    end process;

    q <= q_int;

end behavioral;

```

With it's appropriate testbench:

Código 8: Code 2: Testbench for exercise 1

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_tff is
end tb_tff;

architecture behavior of tb_tff is

    component t_flipflop
        port(
            clk : in std_logic;
            t   : in std_logic;
            q   : out std_logic
        );
    end component;

    signal clk : std_logic := '0';
    signal t   : std_logic := '0';
    signal q   : std_logic;

begin

    uut: t_flipflop
        port map(
            clk => clk,
            t   => t,
            q   => q
        );

    — Generador de reloj
    clk_process : process
    begin
        clk <= '0';
        wait for 5 ns;
        clk <= '1';
        wait for 5 ns;
    end process;

```

```

end process;

stim_proc : process
begin
    -- CLK=0 (no cambia)
    t <= '0';
    wait for 10 ns;

    -- T=0 (mantiene)
    t <= '0';
    wait for 10 ns;

    -- T=1 (toggle)
    t <= '1';
    wait for 10 ns;

    -- Otro toggle
    t <= '1';
    wait for 10 ns;

    wait;
end process;

end behavior;

```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

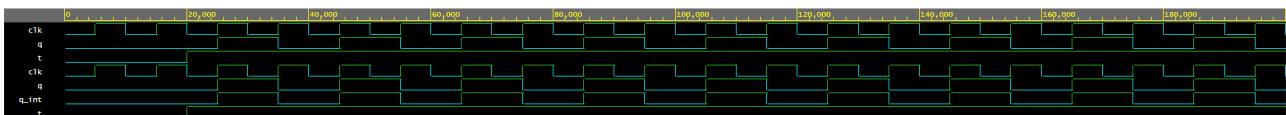


Figure 7: Wave generated for the T flip-flop.

Finally, the code was synthesized and implemented in the Basys 3 board, obtaining the expected results.

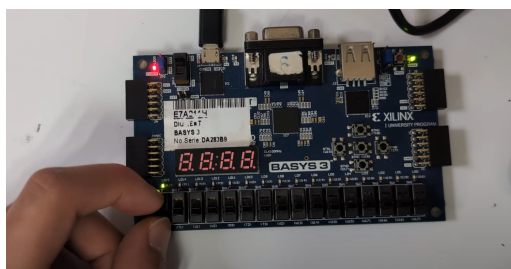


Figure 8: Evidence of the functioning of the T flip-flop in the Basys 3 board.

6 Analysis of Results

In this laboratory practice, four fundamental digital circuits were analyzed—half-subtractor, 2:1 multiplexer, 1→2 demultiplexer, and 1-bit magnitude comparator—following three stages: logical analysis through truth tables, simulation in EDA Playground using VHDL, and practical verification on the Basys 3 board. The observed behavior in each phase is described in detail below.

6.1 Half-Subtractor

6.1.1 Truth Table Analysis

The half-subtractor possesses inputs A (minuend) and B (subtrahend) and generates two outputs: R , representing the logical difference ($A - B$), and AC , which indicates whether a borrow was required. When both inputs are zero, no difference or borrow occurs, and both outputs remain inactive. If $A = 0$ and $B = 1$, subtracting a larger number from a smaller one yields a difference (R) equal to one and activates the borrow ($AC = 1$), reflecting the necessity of borrowing from the higher-order bit. When $A = 1$ and $B = 0$, no borrow is required, and the difference equals one, activating only R . Finally, when $A = 1$ and $B = 1$, the inputs are equal, the subtraction results in zero, and no output is activated. The observed behavior confirms that R functions as an XOR operation and AC as an AND operation with inversion of the minuend.

6.1.2 EDA Playground Analysis

During simulation, the generated waveforms matched the expected values precisely. The R output activated only when the inputs differed, while AC activated exclusively when A was less than B . State transitions occurred without delays or interference, validating the correct implementation of the logical equations in the VHDL code.

6.1.3 Basys 3 Implementation

On the Basys 3 board, switches were employed to represent inputs A and B , while LEDs displayed outputs AC and R . Physical testing demonstrated that the LED corresponding to R illuminated when the inputs differed, and AC lit solely when subtracting 1 from 0. The observed behavior fully coincided with simulation and theoretical expectations, demonstrating correct and stable implementation.

6.2 2:1 Multiplexer

6.2.1 Truth Table Analysis

The 2:1 multiplexer features three inputs: the selection signal Se and data inputs $I1$ and $I0$. Its output, out , reflects the value of one of the two inputs depending on the state of Se . When $Se = 0$, the output adopts the value of $I0$, disregarding $I1$. Conversely, when $Se = 1$, the output assumes the value of $I1$. This behavior confirms that the output is solely dependent on the selector.

6.2.2 EDA Playground Analysis

During simulation, the waveforms exhibited precise switching between data inputs as Se varied. When the selector was low, the output followed $I0$; when high, it followed $I1$. No overshoots or indeterminate signals were observed, validating the correct VHDL implementation.

6.2.3 Basys 3 Implementation

In the physical test, the board switches controlled Se , $I1$, and $I0$, and the LED displayed the output. The observed results aligned with theoretical expectations: with the selector at 0, the output reproduced $I0$; with the selector at 1, it reproduced $I1$. The response was immediate, without switching errors, confirming the accuracy of the logical design and proper pin assignment.

6.3 1→2 Demultiplexer

6.3.1 Truth Table Analysis

The demultiplexer receives the selection signal Se and data input $I0$, generating two outputs: $O1$ and $O0$. When $Se = 0$, the input data is directed exclusively to $O0$, while $O1$ remains low. When $Se = 1$, the signal is transferred to $O1$, and $O0$ is deactivated. This confirms that the circuit directs the input to a single active output based on the selector.

6.3.2 EDA Playground Analysis

Simulation verified that both outputs were never active simultaneously. When the selector changed, the active output correctly switched between $O0$ and $O1$ according to input $I0$. Waveforms demonstrated clean transitions, free of residual signals or timing errors.

6.3.3 Basys 3 Implementation

For the physical implementation, switches were assigned to Se and $I0$, and LEDs indicated outputs $O1$ and $O0$. Testing confirmed that when the selector was low, only $O0$ responded to the input; when high, the response transferred to $O1$. These results corroborated the correct distribution of the signal and the fidelity of the logical design relative to the simulation.

6.4 1-bit Magnitude Comparator

6.4.1 Truth Table Analysis

The magnitude comparator receives two input bits, A and B , and produces three outputs: $A > B$, $A = B$, and $A < B$. When both inputs are zero, they are equal, activating the $A = B$ output. If $A = 0$ and $B = 1$, A is smaller, activating $A < B$. If $A = 1$ and $B = 0$, A is greater, activating $A > B$. Finally, when both inputs are 1, $A = B$ is activated again. In all cases, only one output remains active, confirming the exclusive nature of the comparator's design.

6.4.2 EDA Playground Analysis

Simulation waveforms exhibited clear and stable behavior. Each output responded exclusively to the corresponding relationship between inputs, without overlapping signals. The circuit reacted instantly to changes in A and B , demonstrating precise logic without coding errors.

6.4.3 Basys 3 Implementation

In the physical test, switches represented inputs A and B , and three LEDs displayed outputs $A > B$, $A = B$, and $A < B$. Observations indicated that each LED activated solely under its corresponding condition: “greater” when A exceeded B , “equal” when inputs matched, and “less” when B exceeded A . The complete agreement between simulation and practice confirmed the correct synthesis and mapping of the comparator.

7 Conclusion

This laboratory practice allowed for the analysis, simulation, and implementation of four fundamental digital circuits: half-subtractor, 2:1 multiplexer, 1→2 demultiplexer, and 1-bit magnitude comparator. By comparing theoretical results, simulations in EDA Playground, and physical verification on the Basys 3 board, it was confirmed that all circuits functioned according to the intended designs. The analysis of truth tables enabled a clear understanding of each circuit’s logical behavior, while digital simulation validated the correct VHDL implementation, showing clean transitions and the absence of timing errors or interference. The Basys 3 implementation demonstrated the fidelity of the design by physically reproducing the expected operations using switches and LEDs, confirming the circuits’ stability and accuracy. In conclusion, this practice reinforced the understanding of fundamental digital logic principles, the importance of verification through simulation, and the relevance of physical implementation to validate theoretical designs. Additionally, it strengthened skills in using digital design tools, such as EDA Playground and FPGA boards, which are essential for developing reliable and efficient digital systems.

References

- [1] M. Mathiarasu. (2024) Flip-flops in electronics. [En línea]. Disponible: <https://www.linkedin.com/pulse/flip-flops-electronics-mathiarasu-m-po8oc/>
- [2] ChipVerify. (2024) Sequential logic. [En línea]. Disponible: <https://www.chipverify.com/digital-fundamentals/sequential-logic>
- [3] Intel Corporation, “Vhdl definition,” https://www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_vhdl.htm, 2017, accessed: 2024-10-27.
- [4] J. Schneider and I. Smalley. (2025) Field programmable gate array. [En línea]. Disponible: <https://www.ibm.com/think/topics/field-programmable-gate-arrays>